

## บทที่ 2

### ทฤษฎีคอมโพเนนต์

#### 2.1 คอมโพเนนต์

คอมโพเนนต์ คือ ส่วนย่อยของระบบที่ไม่ขึ้นกับส่วนอื่นๆ และซ่อนรายละเอียดการทำงานไว้ภายใน โดยมีส่วนติดต่อกับภายนอกเพื่อให้สามารถเรียกใช้งานได้ ความสามารถนี้ทำให้คอมโพเนนต์สามารถนำ กลับ มาใช้ใหม่ได้ (reuse)

การพัฒนาเชิงคอมโพเนนต์นั้น มีความหมายกับคนแต่ละกลุ่มแตกต่างกันไปตามความต้องการ ซึ่งมีลักษณะเด่นบางอย่าง ที่นิยามความเป็นระบบคอมโพเนนต์ดังต่อไปนี้

1. คอมโพเนนต์มีลักษณะของออบเจกต์ คือ การซ่อนรายละเอียด (Encapsulated), โพลิมอร์ฟิก (polymorphic), การกำหนดหน้าที่ และการกำหนดอินเทอร์เฟซ
2. คอมโพเนนต์ออกแบบภายใต้เฟรมเวิร์ก (Framework) ซึ่งได้สร้างข้อจำกัดบางอย่างไว้ เช่น ต้องไม่มีหลายเชรต, ไม่มีการติดต่อกับภายนอกโดยไม่ผ่านบริการของเฟรมเวิร์ก
3. คอมโพเนนต์สามารถอยู่ได้ด้วยตัวเอง โดยไม่พึ่งพาคอมโพเนนต์อื่นๆ ยกเว้น คอมโพเนนต์ของเฟรมเวิร์ก ที่คอมโพเนนต์ดังกล่าวใช้อยู่
4. ทุกคอมโพเนนต์มีอินเทอร์เฟซสามัญที่แน่นอน (Fix and common) และอินเทอร์เฟซนี้เปลี่ยนแปลงไม่ได้
5. คอมโพเนนต์สามารถอธิบายตนเองได้โดยอินเทอร์เฟซของคอมโพเนนต์จะต้องมีข้อมูลมากพอ ที่สามารถทำให้ไคลเอนต์สามารถเข้าใจวิธีใช้คอมโพเนนต์นั้นได้

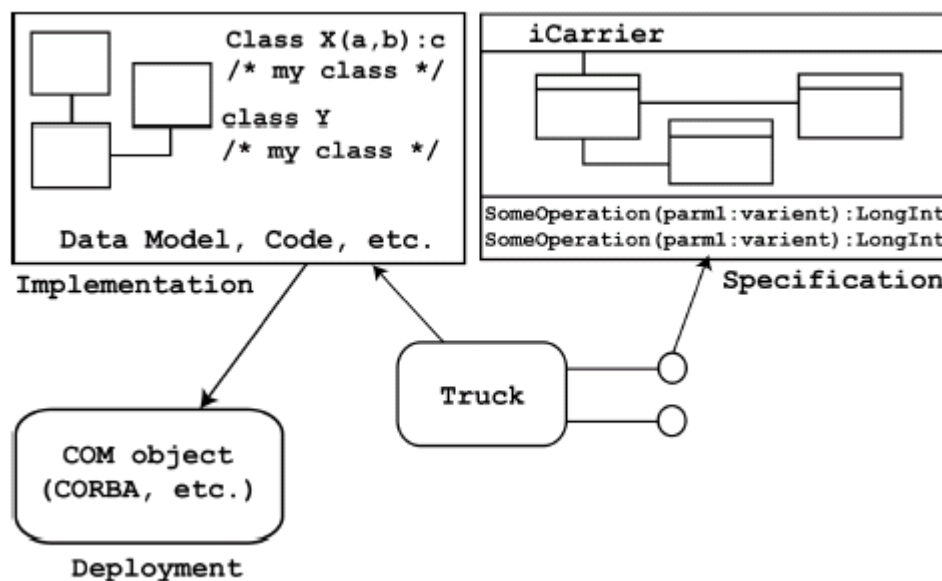
##### 2.1.1 เฟรมเวิร์ก คือ สภาพแวดล้อมที่มีลักษณะดังต่อไปนี้

1. สร้างอินสแตนซ์ของคอมโพเนนต์คอนรันไทม์
2. ทำให้คอมโพเนนต์ค้นพบคอมโพเนนต์อื่นๆ ได้
3. ทำให้คอมโพเนนต์ต่างๆ สามารถติดต่อกันได้
4. จัดหาบริการสามัญ เช่น เพอร์ซิสเทนซ์ (persistence), ทรานส์แอ็กชัน (transaction), ความไม่ขึ้นกับสถานที่, การรักษาความปลอดภัย, มอนิเตอร์ริง (monitoring)

##### 2.1.2 ส่วนประกอบของคอมโพเนนต์

คอมโพเนนต์จะประกอบด้วยส่วนสำคัญ 3 ส่วนดังนี้

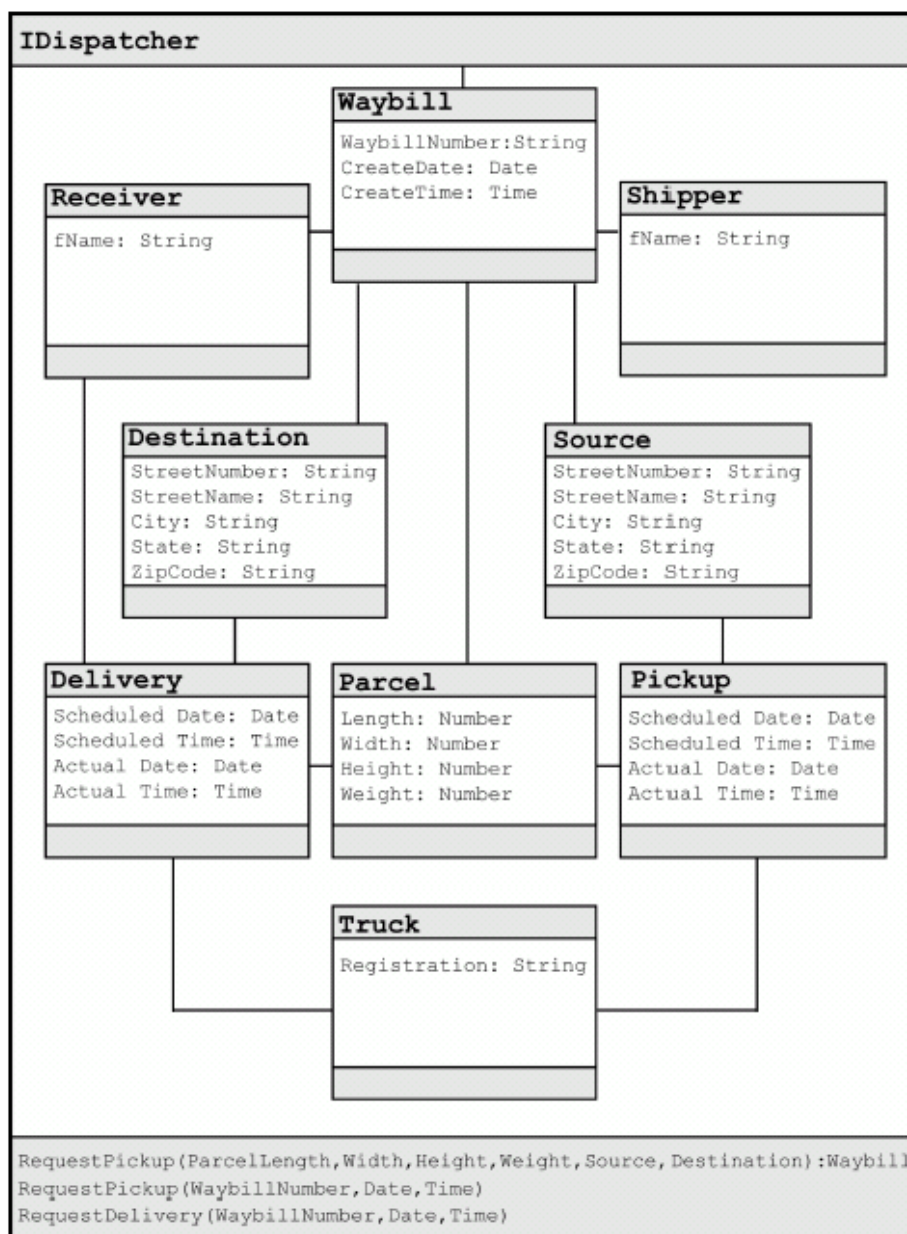
- อินเทอร์เฟซ (interface) - คอมโพเนนต์จะถูกเรียกใช้งานผ่านทางอินเทอร์เฟซ
- อิมพลีเมนต์เชชัน (implementation) - เป็นโค้ดที่กำหนดการทำงานของคอมโพเนนต์
- ดีพลอยเมนต์ (deployment) – เป็นเอ็กซิคิวต์ไฟล์จะใช้ในการทำให้คอมโพเนนต์ทำงานได้ ทำหน้าที่จัดหารันไทม์เอ็นไวรอนเมนต์ในการควบคุมการทำงานของคอมโพเนนต์และจัดหาเซอร์วิสที่จำเป็น



รูปที่ 2-1 คอมโพเนนต์และส่วนประกอบต่าง ๆ ของมัน

### 2.1.3 คุณสมบัติของคอมโพเนนต์

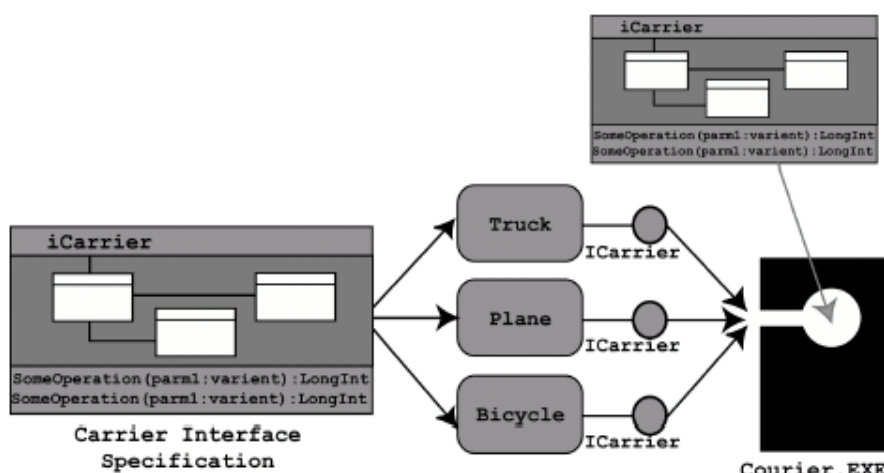
- **Encapsulated** - เป็นกระบวนการในการซ่อนโค้ดหรืออิมพลีเมนต์เดชันของคอมโพเนนต์ ผู้ใช้จะรู้เพียง อินเทอร์เฟซและจะใช้งานผ่านมัน โดยไม่จำเป็นต้องรู้ถึงอิมพลีเมนต์เดชันหรือการทำงาน ทำให้ การเปลี่ยนแปลงอิมพลีเมนต์เดชัน ไม่มีผลกระทบต่อผู้ใช้
- **Descriptive** - เนื่องจากคอมโพเนนต์จะต้องติดต่อผ่านอินเทอร์เฟซเท่านั้น มันจะต้องมีอินฟอร์เมชันของ มันเอง ที่ผู้ใช้สามารถจะเข้าใจได้ โดยอินฟอร์เมชันนั้น จะต้องอธิบายถึง อินเทอร์เฟซ, อิมพลีเมนต์เดชัน และดีพลอยเมนต์



รูปที่ 2-2 ตัวอย่างของอินเทอร์เฟซ

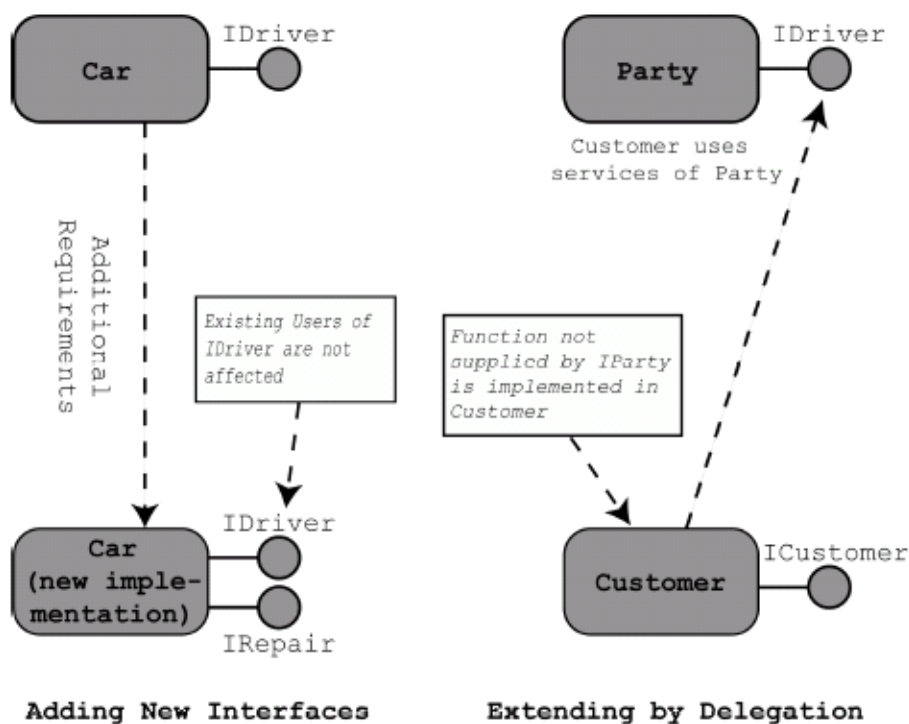
- **Replaceable**

คอมพิวเตอร์มีความสามารถในการเปลี่ยนคอมพิวเตอร์หนึ่งกับคอมพิวเตอร์ ใดๆ ที่มีอินเทอร์เฟซเหมือนกันได้



รูปที่ 2-3 คอมโพเนนต์และการแทนที่

- **Extensible** - สามารถที่จะเพิ่มความสามารถได้ มี 2 วิธีคือ การเพิ่มอินเทอร์เฟซและมอบหมายหน้าที่ (Delegating Responsibility) การสร้างคอมโพเนนต์ใหม่สามารถมอบหมายหน้าที่ให้กับบริการที่มีอยู่ในคอมโพเนนต์ที่มีอยู่แล้ว



รูปที่ 2-4 การเพิ่มเติมคอมโพเนนต์

## 2.2 งานที่เหมาะสมกับการพัฒนาเชิง คอมโพเนนต์

การพัฒนาเชิงคอมโพเนนต์ เป็นวิธีการพัฒนาคอมโพเนนต์โดยใช้คอมโพเนนต์เฟรมเวิร์ก ซึ่งการคงหน้าที่ของคอมโพเนนต์ไว้อย่างถูกต้อง (Well defined responsibility) กระทำได้โดย ให้คอมโพเนนต์เชิงธุรกิจ หลีกเลี่ยงโค้ดที่ไม่สามารถทำงานกับอินเทอร์เฟซที่แน่นอนได้

เฟรมเวิร์กคอมโพเนนต์จะซ่อนบริการต่างๆ ของเอ็นไวรอนเมนต์ ซึ่งบริการต่างๆ เหล่านี้เป็นมาตรฐานแล้ว เช่น message oriented middle-ware, Transaction Monitors, การรักษาความปลอดภัย และ พิสูจน์สิทธิ์

## 2.3 ประโยชน์ของการพัฒนาเชิงคอมโพเนนต์

การพัฒนาเชิงคอมโพเนนต์มีประโยชน์อย่างมากด้านเกี่ยวกับเทคนิค และประโยชน์เหล่านี้ ยังนำไปสู่ประโยชน์ด้านธุรกิจด้วย

### ประโยชน์ด้านเทคนิค

1. จัดการความซับซ้อนได้ง่ายขึ้น ทำให้คุณภาพของการแก้ปัญหาซอฟต์แวร์ง่ายขึ้น
2. งานซับซ้อนที่ไม่เกี่ยวกับฟังก์ชันธุรกิจ ถูกรวมไว้ในเฟรมเวิร์ก
3. การออกแบบ, อิมพลีเมนต์, ทดสอบ ส่วนต่างๆ อย่างอิสระ ทำให้สามารถพัฒนาส่วนต่างๆ ได้ พร้อมกัน
4. การไม่ขึ้นต่อกันของคอมโพเนนต์ลดผลกระทบของการเปลี่ยนแปลง requirement ไม่ให้กระจาย ไปทั้งระบบ

### ประโยชน์ด้านธุรกิจ

1. ผลิตภัณฑ์มีคุณภาพมากขึ้น
2. เร่งเวลาออกสู่ตลาด
3. ใช้ทรัพยากรบุคคลได้คุ้มค่าขึ้น
4. ลดค่าใช้จ่าย
5. เพิ่มปริมาณการนำกลับมาใช้ ในโครงการต่อไป